Introduction to C++

Outline

- Introduction
- Main characteristics of the programming language
- Variables and data types
- Input-output stream
- Operators
- Conditional statements
- Loops
- Arrays
- The `main` and custom functions
- A complete algorithm: the maximum value in an array
- Bitwise operators
- Key points of the chapter

Introduction

Introduction: from pseudo-code to C++

- In the chapter «Pseudo-code» algorithms are written in a language meant for humans
- It is precise, but it is **not executed** by a computer
- To run an algorithm we have to translate it into a **programming language**
- In this chapter we introduce the programming language C++, which we use to implement the algorithms of the course

The path of the chapter

The chapter follows the same path as the chapter «Pseudo-code»:

- the main characteristics of the language and the structure of a program
- variables, data types and the input-output stream
- operators, conditional statements and loops
- arrays, whose size can be fixed or decided during the execution
- functions, with the `main` function as the entry point of every program
- a complete algorithm, the search of the maximum value in an array, written in pseudo-code and in C++
- the bitwise operators, which act directly on the binary representation of integer numbers

Every fragment of code has been executed

Every fragment of code of the chapter
has been compiled with the g++ compiler (standard C++17)
and executed.

The outputs reported in the examples
are the ones printed by the programs.

Indices: pseudo-code and C++

- In pseudo-code the entries of an array $a[1..n]$ are indexed from 1 to n
- In `C++` the entries of an array of size n are indexed from 0 to $n-1$
- The entry $a[i]$ of the pseudo-code corresponds to the entry `a[i-1]` of the `C++` code

Summary

- Pseudo-code is precise but is not executed: to run an algorithm we write it in a programming language, here C++
- The chapter follows the path of the chapter «Pseudo-code», up to a complete algorithm and the bitwise operators
- Pseudo-code indices go from 1 to n, C++ indices from 0 to $n - 1$

Main characteristics of the programming language

Introduction: main characteristics of C++

- We first describe what kind of language C++ is
- Then we look at the skeleton that every C++ program shares, and at how a program is compiled and executed
- This is the frame in which all the instructions of the following sections are written

Main characteristics (1/2)

- C++ is an **imperative** language that gives direct access to the memory and to the hardware resources of the computer
- It is highly efficient for low-level system programming while also supporting high-level abstractions
- This makes it versatile for both performance-critical applications and modern software development
- C++ supports **functions**, which enable a structured and modular approach to programming
- Functions allow code reuse and better organization

Main characteristics (2/2)

- C++ includes a predefined set of **reserved symbols** used to declare variables, define functions and specify executable instructions
- The **source code** consists of a sequence of **atomic operations** executed in order, one after the other, operating on variables of specific **data types**
- The source code can contain **comments**, which serve as documentation for the code and are ignored by the compiler
- A single-line comment starts with the reserved symbol "//"
- Multi-line comments are enclosed between the reserved symbols "/*" and "*/"

Example: comments in C++

This is an example of a single-line comment:

```
1 // This is a comment line
```

This is an example of a multi-line comment:

```
1 /* This is a
2    multi-line comment */
```

The structure of a program

- Before describing the single instructions, we look at the skeleton that every program shares
- A `C++` program is a text file (with extension `.cpp`) that is translated by a **compiler** into an executable file
- The execution of the program starts from the `main` function, the entry point of every program

Example: the skeleton of a program (1/2)

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      // The instructions of the program are written here
8      cout << "Hello, world!" << endl;
9
10     return 0;
11 }
```

Example: the skeleton of a program (2/2)

If the source code is saved in the file `hello.cpp`, it is compiled and executed with the commands

```
g++ -Wall -std=c++17 -o hello hello.cpp
./hello
```

and the output is

```
Hello, world!
```

Fragments of code

In the following examples we write
only the instructions that matter.

Unless they contain a complete program,
the fragments of code are meant to be placed
inside the curly braces of `main`.

Every instruction ends with a semicolon

Each instruction of a C++ source code
must end with the reserved symbol “;”.

Summary

The language:

- C++ is imperative, efficient, and supports functions
- The source code is a sequence of atomic operations on variables of specific data types
- Comments: "//" for one line, "/* ... */" for several lines

The structure of a program:

- A .cpp file is compiled (for example with g++) into an executable file
- The execution starts from `main`; every instruction ends with ";"

Variables and data types

Introduction: variables and data types

- The source code operates on **variables**, and every variable has a **data type**
- We see the fundamental data types of C++ and how variables are declared and assigned
- Every algorithm of the course stores its data in variables of these types

Data types (1/2)

Information is categorized into different **data types**. The four fundamental families of data types are:

1. The data type "`int`" represents an integer, which is a whole number written without a decimal point. This includes positive and negative whole numbers, as well as zero.

2. The data type "`bool`" represents a Boolean value: `true` or `false`.

Data types (2/2)

3. The data type "`float`" represents a floating-point number, which includes numbers written with a decimal point. The data type "`double`" represents a floating-point number with double precision, providing greater accuracy and range than `float`.

4. The data type "`char`" represents a single character, such as a letter, digit, or symbol, enclosed in single quotes (e.g., `'A'`, `'7'`, `'!'`).

- A sequence of characters enclosed in double quotes (e.g., `"Ada"`) is stored in a variable of type "`string`", provided by the standard library `<string>`

Variables

- A variable acts as a container that stores a value of a specific data type for later use
- Each variable has a unique name and holds a single value at any given time
- Variables are assigned values using the assignment operator "="
- When a new value is assigned, the previous value is overwritten

Declaration before use

Variables must be declared before use,
specifying their data type.

Example: variable declarations and value assignments (1/2)

```
1  int age = 25; // Integer variable
2  bool isRaining = false; // Boolean variable
3  float smallNumber = 3.14; // Floating-point variable
4  double largeNumber = 123456.789; // Double precision
       floating-point variable
5  char character = 'a'; // Char variable
6  string name = "Ada"; // String variable
7
8  age = 26; // The previous value 25 is overwritten
9  cout << age << " " << isRaining << " " << smallNumber
       << " "
10       << largeNumber << " " << character << " " << name
       << endl;
```

Output:

```
26 0 3.14 123457 a Ada
```

Example: variable declarations and value assignments (2/2)

- The Boolean value `false` is displayed as `0`
- The number `123456.789` is displayed as `123457`: by default, `cout` displays at most six significant digits
- The value stored in the variable does not change, and the number of displayed digits can be chosen, as we see now

Summary

Data types:

- `int`, `bool`, `float` and `double`, `char`; strings are of type `string`

Variables:

- A variable stores one value of its data type; "=" assigns a new value and overwrites the previous one
- Variables must be declared, with their data type, before use
- `cout` displays `false` as `0` and at most six significant digits

Input-output stream

Introduction: input-output stream

- To display the values of the variables on the screen, a program uses the **input-output stream** library
- We see how to include it, how to print text and values, and how to choose the number of displayed digits
- Printing is how we see the results of our algorithms

The input-output stream library

- The include "`#include <iostream>`" allows to use the input-output stream library in C++

Example: include

```
1 #include <iostream>
```

The output stream

- The values of variables can be displayed as output text on the screen using the output stream "`std::cout`"
- The insertion operator "<<" is used to send data to the output stream
- Multiple values can be displayed in a single `cout` statement by chaining them with the "<<" operator
- Text should be enclosed in double quotes "`"`", while variable values are written directly
- The manipulator "`std::endl`" is used with `std::cout` to insert a new line on the screen and flush the output buffer

Example: screen printings

```
1 int n = 7;
2 std::cout << "The value of the variable is " << n <<
      std::endl;
```

Output:

```
The value of the variable is 7
```

Namespace and number of decimals

- The statement "`using namespace std;`" simplifies the use of standard library functions by eliminating the need to prefix them with `std::`
- From now on we always use it, and we write `cout` and `endl`
- The include "`#include <iomanip>`" gives the manipulators "`fixed`" and "`setprecision(k)`"
- When they are written together in a `cout` statement, they display the floating-point numbers that follow with exactly k digits after the decimal point

Summary

- `#include <iostream>` gives the input-output stream library
- `cout` displays text (in double quotes) and values, chained with "<<"; `endl` inserts a new line
- `using namespace std;` avoids the prefix `std::`
- With `#include <iomanip>`, `fixed` and `setprecision(k)` display k digits after the decimal point

Operators

Introduction: operators

- Operators are symbols that perform operations on variables and values
- According to their functionality, they are classified into five main categories:
  - arithmetic operators
  - assignment operators
  - increment and decrement operators
  - relational operators
  - logical operators
- For each category we see the list of operators and an example with its output

Arithmetic operators

Arithmetic operators, used to perform basic mathematical operations:

- + (Addition) $\rightsquigarrow$ Adds two values.
- - (Subtraction) $\rightsquigarrow$ Subtracts one value from another.
- * (Multiplication) $\rightsquigarrow$ Multiplies two values.
- / (Division) $\rightsquigarrow$ Divides one value by another.
- % (Modulus) $\rightsquigarrow$ Returns the remainder of a division.

Example: arithmetic operators

```
1 int a = 10, b = 3;
2
3 cout << "Addition: " << (a + b) << endl;
4 cout << "Subtraction: " << (a - b) << endl;
5 cout << "Multiplication: " << (a * b) << endl;
6 cout << "Division: " << (a / b) << endl; // Integer
      division
7 cout << "Modulus: " << (a % b) << endl; // Remainder of
       division
```

Output:

```
Addition: 13
Subtraction: 7
Multiplication: 30
Division: 3
Modulus: 1
```

Integer division and type conversion

- The division operator `/` performs integer division when used with integers: the result `10 / 3` is 3
- To obtain a floating-point result, at least one operand must be of data type `float` or `double`
- For example, `10.0 / 3` returns `3.33333` instead of 3
- A value of type `int` is converted into a value of type `double` by writing "`(double)`" before it (**type conversion**)
- For example, if `a` and `b` are variables of type `int`, the expression `(double)a / b` performs the floating-point division

Example: integer and floating-point division

```
1 int a = 10, b = 3;
2
3 cout << a / b << endl;              // Integer division: 3
4 cout << 10.0 / 3 << endl;          // Floating-point
      division
5 cout << (double)a / b << endl;     // Conversion of a to
      double
6 cout << fixed << setprecision(2) << (double)a / b <<
      endl; // Two decimals
```

Output (the last line requires `#include <iomanip>`):

```
3
3.33333
3.33333
3.33
```

Assignment operators

Assignment operators, used to assign values to variables:

- = (Assignment) $\rightsquigarrow$ Assigns a value to a variable.
- += (Addition assignment) $\rightsquigarrow$ Adds a value to a variable and assigns the result.
- -= (Subtraction assignment) $\rightsquigarrow$ Subtracts a value from a variable and assigns the result.
- *= (Multiplication assignment) $\rightsquigarrow$ Multiplies a variable by a value and assigns the result.
- /= (Division assignment) $\rightsquigarrow$ Divides a variable by a value and assigns the result.
- %= (Modulus assignment) $\rightsquigarrow$ Applies the modulus operation and assigns the result.

Example: assignment operators (1/2)

```
1  int x = 10;  // Assignment
2  cout << "Initial value: " << x << endl;
3
4  x += 5;  // Equivalent to: x = x + 5
5  cout << "After += 5: " << x << endl;
6
7  x -= 3;  // Equivalent to: x = x - 3
8  cout << "After -= 3: " << x << endl;
9
10 x *= 2;  // Equivalent to: x = x * 2
11 cout << "After *= 2: " << x << endl;
12
13 x /= 4;  // Equivalent to: x = x / 4
14 cout << "After /= 4: " << x << endl;
15
16 x %= 3;  // Equivalent to: x = x % 3
17 cout << "After %= 3: " << x << endl;
```

Example: assignment operators (2/2)

Output:

```
Initial value: 10
After += 5: 15
After -= 3: 12
After *= 2: 24
After /= 4: 6
After %= 3: 0
```

Increment and decrement operators

Increment and decrement operators:

- ++ (Increment) $\rightsquigarrow$ Increases the value of a variable by 1.
- -- (Decrement) $\rightsquigarrow$ Decreases the value of a variable by 1.

The increment operator ++ can be used in two forms:

- pre-increment ("++x"): increments the value before returning it;
- post-increment ("x++"): returns the value first, then increments it.

The same logic applies to the decrement operator (--x and x--).

Example: increment and decrement operators (1/2)

```
1  int x = 5;
2
3  cout << "Initial value: " << x << endl; // Displays 5
4
5  cout << "Pre-increment (++x): " << ++x << endl; //
       Displays 6
6
7  cout << "Post-increment (x++): " << x++ << endl; //
       Displays 6
8  cout << "Value after post-increment: " << x << endl; //
        Displays 7
9
10 cout << "Pre-decrement (--x): " << --x << endl; //
       Displays 6
11
12 cout << "Post-decrement (x--): " << x-- << endl; //
       Displays 6
13 cout << "Value after post-decrement: " << x << endl; //
        Displays 5
```

Example: increment and decrement operators (2/2)

Output:

```
Initial value: 5
Pre-increment (++x): 6
Post-increment (x++): 6
Value after post-increment: 7
Pre-decrement (--x): 6
Post-decrement (x--): 6
Value after post-decrement: 5
```

Relational operators

Relational operators, used to compare values and return a Boolean result:

- == (Equal to) $\rightsquigarrow$ Checks if two values are equal.
- != (Not equal to) $\rightsquigarrow$ Checks if two values are different.
- < (Less than) $\rightsquigarrow$ Checks if one value is smaller than another.
- > (Greater than) $\rightsquigarrow$ Checks if one value is larger than another.
- <= (Less than or equal to) $\rightsquigarrow$ Checks if one value is smaller than or equal to another.
- >= (Greater than or equal to) $\rightsquigarrow$ Checks if one value is larger than or equal to another.

Boolean results

Relational operators return a Boolean result:

- `true` (displayed as `1`) if the condition is satisfied;
- `false` (displayed as `0`) if the condition is not satisfied.

These operators are commonly used in **conditional statements** and **loops**.

Example: relational operators (1/2)

```
1 int a = 10, b = 5;
2
3 cout << "a == b: " << (a == b) << endl;  // Equal to
4 cout << "a != b: " << (a != b) << endl;  // Not equal
      to
5 cout << "a < b: " << (a < b) << endl;    // Less than
6 cout << "a > b: " << (a > b) << endl;    // Greater
      than
7 cout << "a <= b: " << (a <= b) << endl;  // Less than
      or equal to
8 cout << "a >= b: " << (a >= b) << endl;  // Greater
      than or equal to
```

Example: relational operators (2/2)

Output:

```
a == b: 0
a != b: 1
a < b: 0
a > b: 1
a <= b: 0
a >= b: 1
```

Logical operators

Logical operators, used for logical operations, often in conditional statements:

- `&&` (Logical AND) $\rightsquigarrow$ Returns `true` if both conditions are true.
- `||` (Logical OR) $\rightsquigarrow$ Returns `true` if at least one condition is true.
- `!` (Logical NOT) $\rightsquigarrow$ Reverses the Boolean value.

Like the relational operators, logical operators return Boolean values, `true` (`1`) or `false` (`0`), and are commonly used in **conditional statements** and **loops**.

Example: logical operators (1/2)

```
1  bool condition1 = true;
2  bool condition2 = false;
3
4  cout << "condition1: " << condition1 << endl;
5  cout << "condition2: " << condition2 << endl;
6
7  // Logical AND: true only if both conditions are true
8  cout << "condition1 && condition2: " << (condition1 &&
       condition2) << endl;
9
10 // Logical OR: true if at least one condition is true
11 cout << "condition1 || condition2: " << (condition1 ||
       condition2) << endl;
12
13 // Logical NOT: reverses the Boolean value
14 cout << "!condition1: " << !condition1 << endl;
15 cout << "!condition2: " << !condition2 << endl;
```

Example: logical operators (2/2)

Output:

```
condition1: 1
condition2: 0
condition1 && condition2: 0
condition1 || condition2: 1
!condition1: 0
!condition2: 1
```

Summary (1/2)

Arithmetic operators:

- `+`, `-`, `*`, `/`, `%`
- Between two `int` values, `/` is the integer division; `(double)` converts an `int` into a `double`

Assignment, increment and decrement operators:

- `x += 5` is equivalent to `x = x + 5`, and similarly for `-=`, `*=`, `/=`, `%=`
- `++x` increments and then returns the value, `x++` returns the value and then increments it

Summary (2/2)

Relational operators:

- ==, !=, <, >, <=, >= compare two values

Logical operators:

- &&, ||, ! combine Boolean values

Boolean results:

- Both return `true` (displayed as `1`) or `false` (displayed as `0`), and are used in conditional statements and loops

Conditional statements

Introduction: conditional statements

- The relational and logical operators produce the conditions that decide which instructions are executed
- We see the `if` statement, with its `else` part, and the ternary operator `?:`, its compact form
- Every algorithm makes choices: conditional statements are how C++ writes them

The `if` statement

- Conditional statements allow to execute different blocks of code based on specific conditions
- The `if` statement is the most fundamental control structure of this type
- The `if` statement evaluates a condition inside parentheses `()`
- If the condition is **true**, the block of code inside the curly braces `{}` is executed
- If the condition is **false**, the program skips the block and executes the block of code inside the curly braces `{}` of the `else` part, if present

Example: conditional statement

```
1  int number = 10;
2
3  // Check if number is positive
4  if (number > 0)
5  {
6      cout << "The number is positive." << endl;
7  }
8  else
9  {
10     cout << "The number is non-positive." << endl;
11 }
```

Output:

```
The number is positive.
```

Ternary operator (conditional operator)

- The **ternary operator** `?:` provides a compact way to write simple `if-else` statements
- It is the only operator in C++ that takes three operands
- Syntax: `condition ? value_if_true : value_if_false`
- If the condition is **true**, the expression evaluates to `value_if_true`; otherwise, it evaluates to `value_if_false`

Example: ternary operator (1/2)

```
1  int number = 10;
2
3  // Using if-else statement
4  string result1;
5  if (number > 0)
6      result1 = "positive";
7  else
8      result1 = "non-positive";
9
10 cout << "The number is " << result1 << endl;
11
12 // Using ternary operator (compact form)
13 string result2 = (number > 0) ? "positive" : "non-
       positive";
14
15 cout << "The number is " << result2 << endl;
```

Example: ternary operator (2/2)

Output:

```
The number is positive
The number is positive
```

Example: finding the maximum of two numbers (1/2)

```
1  int a = 15, b = 20;
2
3  // Using if-else statement
4  int max1;
5  if (a > b)
6      max1 = a;
7  else
8      max1 = b;
9
10 cout << "Maximum (if-else): " << max1 << endl;
11
12 // Using ternary operator (compact form)
13 int max2 = (a > b) ? a : b;
14
15 cout << "Maximum (ternary): " << max2 << endl;
```

Example: finding the maximum of two numbers (2/2)

Output:

```
Maximum (if-else): 20
Maximum (ternary): 20
```

Example: checking even or odd

```
1 int n = 7;
2
3 // Using ternary operator
4 string parity = (n % 2 == 0) ? "even" : "odd";
5
6 cout << n << " is " << parity << endl;
```

Output:

```
7 is odd
```

Example: assigning the absolute value

```
1 int x = -5;
2
3 // Using ternary operator to compute absolute value
4 int absValue = (x >= 0) ? x : -x;
5
6 cout << "Absolute value of " << x << " is " << absValue
      << endl;
```

Output:

```
Absolute value of -5 is 5
```

Example: nested ternary operators (1/2)

```
1 int score = 85;
2
3 // Nested ternary operators to assign a grade
4 string grade = (score >= 90) ? "A" :
5                (score >= 80) ? "B" :
6                (score >= 70) ? "C" :
7                (score >= 60) ? "D" : "F";
8
9 cout << "Score: " << score << ", Grade: " << grade <<
      endl;
```

Output:

```
Score: 85, Grade: B
```

Example: nested ternary operators (2/2)

- The conditions are evaluated from top to bottom
- `score >= 90` is false and `score >= 80` is true, so the grade is `"B"`
- For readability, complex conditions are better handled with `if-else` statements

Summary

The `if` statement:

- `if (condition) {...}` executes the block when the condition is true
- The block of the `else` part, if present, is executed when the condition is false

The ternary operator:

- `condition ? value_if_true : value_if_false` is the compact form of a simple `if-else`
- Useful for the maximum of two numbers, the parity, the absolute value
- Nested ternary operators are evaluated from top to bottom; complex conditions are better written with `if-else`

Loops

Introduction: loops

- Conditional statements choose *whether* a block of code is executed; loops decide *how many times* it is executed
- We see the `while` loop, the `for` loop, and the statements `break` and `continue`
- Almost every algorithm of the course repeats some instructions: loops are its basic tool

Loops

- Loops are used to execute a block of code repeatedly while a condition is met
- They are essential for automating repetitive tasks and reducing redundant code
- In C++, the most commonly used loops are the `while` loop and the `for` loop

The `while` loop

- The `while` loop executes a block of code as long as the condition inside the parentheses `()` evaluates to **true**
- When the condition is false, the loop terminates

Example: `while` loop

```
1 int i = 1; // Initialization
2
3 while (i <= 5) // Condition
4 {
5     cout << "Number: " << i << endl;
6     i++; // Increment
7 }
```

Example: `while` loop

Output:

```
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
```

The condition `i <= 5` is evaluated six times: five times it is true, and the sixth time (`i` equal to 6) it is false and the loop terminates.

The `for` loop

- The `for` loop is a compact loop structure that combines initialization, condition, and increment (or decrement) in one statement

Example: `for` loop

```
1 for (int i = 1; i <= 5; i++) // Initialization,
    condition, increment
2 {
3     cout << "Number: " << i << endl;
4 }
```

The output is the same as in the `while` loop of the previous example.

The `break` and `continue` statements

- The `break` statement immediately terminates the loop
- The `continue` statement skips the rest of the current iteration and moves to the next one

Example: break and continue (1/2)

```
1  for (int i = 1; i <= 10; i++)
2  {
3      if (i % 2 == 0)
4      {
5          continue; // Skip the even numbers
6      }
7      if (i > 7)
8      {
9          break; // Stop the loop at the first odd number
       greater than 7
10     }
11     cout << i << " ";
12 }
13 cout << endl;
```

Example: `break` and `continue` (2/2)

Output:

```
1 3 5 7
```

- The even values of `i` are skipped by `continue`
- When `i` is equal to 9 the `break` statement terminates the loop, and the value 10 is never reached

Summary

Loops:

- `while (condition) {...}` repeats the block as long as the condition is true
- `for (initialization; condition; increment) {...}` puts the three parts in one statement

Interrupting a loop:

- `break` terminates the loop immediately
- `continue` skips the rest of the current iteration and moves to the next one

Arrays

Introduction: arrays

- Loops become really useful when they visit collections of values
- A collection of variables of the same data type is stored in an **array**
- We see fixed-size arrays, whose values are known when the program is written, and dynamic-size arrays, whose size is decided during the execution
- The input of most algorithms of the course is an array

Fixed-size arrays

- An array whose values are known when the program is written can be declared by listing its values inside curly braces
- The size of the array is the number of listed values, and it cannot change during the execution
- Each element of an array is accessed using its **index**, starting from zero, inside square brackets "`[]`"
- The elements of an array of size n are `b[0]`, `b[1]`, ..., `b[n-1]`

Example: fixed-size arrays

```
1  int b[] = {9, 12, 3, 16}; // Array of 4 integers
2  int n = 4;
3
4  b[2] = 5; // Assign a new value to the third element
5
6  for (int i = 0; i < n; i++)
7  {
8      cout << b[i] << " ";
9  }
10 cout << endl;
```

Output:

```
9 12 5 16
```

Two-dimensional fixed-size arrays

- A two-dimensional fixed-size array is declared with two sizes, the number of rows and the number of columns
- Its values are listed row by row
- `int c[2][3] = {{3, 1, 4}, {1, 5, 9}};` declares an array with 2 rows and 3 columns
- `c[1][2]` is the element in the second row and third column, equal to 9

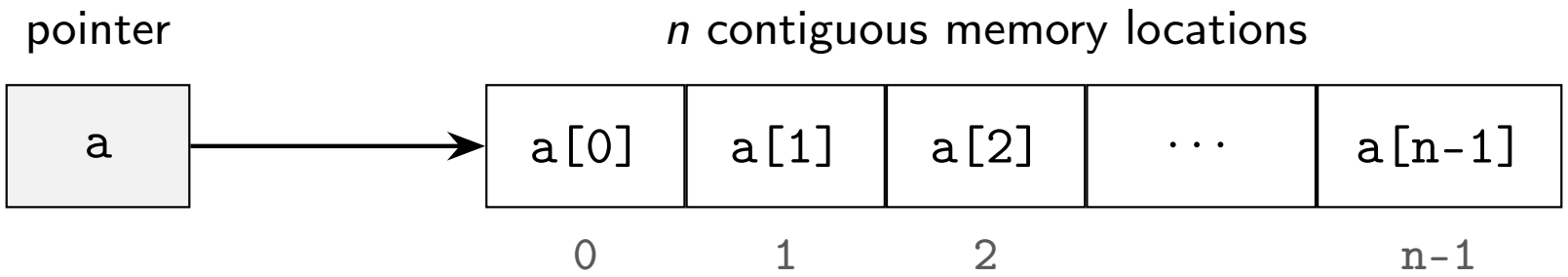
Dynamic-size arrays (1/2)

- A **dynamic-size array** is a collection of variables of the same data type, stored in contiguous memory locations
- The size of the array is determined during the execution of the algorithm
- The reserved symbol "*" is used to declare a **pointer**, which stores the memory address of the first element of a dynamic-size array
- The `new` operator is used to allocate the memory of a dynamic-size array, and it must specify the data type of the elements along with the size of the array inside square brackets "[]"

Dynamic-size arrays (2/2)

- Each element of a dynamic-size array is accessed using its index, starting from zero, inside square brackets "`[]`"
- A dynamic-size array requires memory deallocation using the `delete[]` operator to free memory and prevent memory leaks

A dynamic-size array in memory



A dynamic-size array of size n: the pointer a stores the address of the first element, and the elements are indexed from 0 to $n-1$.

Example: dynamic-size arrays (1/2)

```
1  int n = 5; // Size of the array
2
3  int* a = new int[n]; // Memory allocation of an array
       of size n
4
5  for (int i = 0; i < n; i++)
6  {
7      a[i] = i * 2; // Example of value assignment
8  }
9
10 for (int i = 0; i < n; i++)
11 {
12     cout << a[i] << " ";
13 }
14 cout << endl;
15
16 delete[] a;  // Free allocated memory
```

Example: dynamic-size arrays (2/2)

Output:

```
0 2 4 6 8
```

Dynamic-size two-dimensional arrays (1/2)

- Dynamic-size arrays can be multidimensional
- For example, the reserved symbol "**" is used to declare a pointer to a pointer, which stores the memory address of the first element of a dynamic-size two-dimensional array
- This is an array of m pointers, each pointing to a row of n elements

Dynamic-size two-dimensional arrays (2/2)

- The `new` operator is also used to allocate the memory of dynamic-size two-dimensional arrays: first the array of the m row pointers, then each row
- Each element of a dynamic-size two-dimensional array is accessed using two indices, both starting from zero, inside square brackets "`[] []`"
- The memory is freed in the reverse order: first each row, then the array of row pointers

Example: dynamic-size two-dimensional arrays (1/3)

Memory allocation:

```
1 int m = 3; // Number of rows
2 int n = 4; // Number of columns
3
4 int** matrix = new int*[m]; // Memory allocation of a
      matrix of size m times n
5 for (int i = 0; i < m; i++)
6 {
7     matrix[i] = new int[n]; // Allocate each row
8 }
```

Example: dynamic-size two-dimensional arrays (2/3)

Value assignment and printing:

```
10 for (int i = 0; i < m; i++)
11 {
12     for (int j = 0; j < n; j++)
13     {
14         matrix[i][j] = i * j; // Example of value
       assignment
15         cout << matrix[i][j] << " ";
16     }
17     cout << endl;
18 }
```

Example: dynamic-size two-dimensional arrays (3/3)

Memory deallocation:

```
20 for (int i = 0; i < m; i++) // Free allocated memory
21 {
22     delete[] matrix[i]; // Free each row
23 }
24 delete[] matrix; // Free row pointers
```

Output:

```
0 0 0 0
0 1 2 3
0 2 4 6
```

Summary

Fixed-size arrays:

- Declared by listing their values, as in `int b[] = {9, 12, 3, 16};`
- Elements `b[0]`, ..., `b[n-1]`; two-dimensional arrays `c[i][j]`

Dynamic-size arrays:

- The size is decided during the execution; the pointer stores the address of the first element
- Allocated with `new`, freed with `delete[]`
- Two-dimensional: an array of row pointers (`**`), each row allocated with `new`, and freed in the reverse order

The `main` and custom functions

Introduction: the `main` and custom functions

- As in pseudo-code, an algorithm is written as a function that receives the input and returns the output
- We see how a function is made, the role of the `main` function, and how the parameters are passed: by value or by reference
- Every algorithm of the course is implemented as a `C++` function

Functions

- Functions allow programmers to write modular and reusable code by grouping instructions into a single unit
- A function can be called multiple times to execute the same task, reducing redundancy and improving code organization
- Functions typically return a value and accept multiple parameters as input
- When calling a function, the arguments are passed to the parameters

The parts of a function

A function in C++ consists of:

1. a **return type**: specifies the type of the value returned by the function (`void` if nothing is returned);
2. a **function name**: the unique identifier of the function;
3. a **parameter list** (optional): specifies the input values of the function;
4. a **body**: contains the block of code executed when the function is called;
5. a **return statement** (optional): used to return a value if the function returns a value.

The `main` function

- The `main` function serves as the entry point of a `C++` program: it is executed first when the program starts running
- Every `C++` program must contain exactly one `main` function
- Its return value `0` tells the operating system that the program terminated correctly
- A function must be defined (or declared) before the point where it is called: for this reason the custom functions are written before `main`
- `C++` source code can also be compiled and executed online, for example at the following address: https://www.mycompiler.io/it/online-c++-compiler

Example: functions (1/2)

```cpp
#include <iostream>

using namespace std;

// Function definition
int addition(int a, int b)
{
    return a + b; // Returns the sum
}

// Function definition
int multiplication(int a, int b)
{
    return a * b; // Returns the product
}
```

Example: functions (2/2)

```
17 int main()
18 {
19     int sum = addition(5, 10); // Function call
20     cout << "Sum: " << sum << endl;
21     int prod = multiplication(5, 10); // Function call
22     cout << "Prod: " << prod << endl;
23
24     return 0;
25 }
```

Output:

```
Sum: 15
Prod: 50
```

Parameters passed by value and by reference

A function returns at most one value.

When a function has to produce more than one result,
or has to modify a variable of the caller,
its parameters are passed by reference.

By value, by reference, arrays

- By default, a parameter is passed **by value**: the function receives a copy of the argument, and the changes made to the copy do not affect the variable of the caller
- A parameter is passed **by reference** by writing the reserved symbol "`&`" after its data type (for example `int& x`): the parameter is another name for the variable of the caller, and every change made by the function is visible to the caller
- An array is passed to a function through the pointer to its first element (for example `int *a`), together with its size: the function works directly on the elements of the array of the caller

Example: parameters passed by value and by reference (1/4)

```
1  #include <iostream>
2
3  using namespace std;
4
5  void incrementByValue(int x)
6  {
7      x = x + 1; // Modifies only the local copy
8  }
9
10 void incrementByReference(int& x)
11 {
12     x = x + 1; // Modifies the variable of the caller
13 }
```

Example: parameters passed by value and by reference (2/4)

```
15 void fillWithZeros(int n, int* a)
16 {
17     for (int i = 0; i < n; i++)
18     {
19         a[i] = 0; // Modifies the array of the caller
20     }
21 }
```

Example: parameters passed by value and by reference (3/4)

```
23 int main()
24 {
25     int k = 5;
26     incrementByValue(k);
27     cout << "After incrementByValue: " << k << endl;
28     incrementByReference(k);
29     cout << "After incrementByReference: " << k << endl
       ;
30
31     int b[] = {4, 7, 1};
32     fillWithZeros(3, b);
33     cout << "Array: " << b[0] << " " << b[1] << " " <<
       b[2] << endl;
34
35     return 0;
36 }
```

Example: parameters passed by value and by reference (4/4)

Output:

```
After incrementByValue: 5
After incrementByReference: 6
Array: 0 0 0
```

Summary

Functions:

- Return type, name, parameter list, body, return statement
- `main` is the entry point, there is exactly one, and it returns `0`; custom functions are written before `main`

Passing the parameters:

- By value: the function works on a copy
- By reference (`int& x`): the changes are visible to the caller
- An array is passed through the pointer to its first element, together with its size

A complete algorithm: the maximum value in an array

Introduction: a complete algorithm

- We now put all the pieces together: a function, a loop, a conditional statement and a dynamic-size array
- We take an algorithm written in pseudo-code and translate it into C++, paying attention to the indices
- This is the path that every algorithm of the course follows, from the pseudo-code to a running program

The problem

We consider the problem of finding the maximum value in an array a of $n \geq 1$ integer numbers.

- The next slide shows the pseudo-code of the algorithm `FindMaximum` of the chapter «Pseudo-code»: the entries are indexed from 1 to n

The pseudo-code

Algorithm: FindMaximum

Input : An array $a[1..n]$ of $n \geq 1$ integer numbers
Output: The maximum value in the array a

```
1 function FindMaximum(a, n)
2 |   max := a[1]
3 |   for i := 2 to n do
4 |   |   if a[i] > max then
5 |   |   |   max ← a[i]
6 |   |   end
7 |   end
8 |   return max
9 end
```

From the pseudo-code to C++

- In the C++ implementation the entries are indexed from 0 to $n - 1$
- So the initialization uses `a[0]` and the loop starts from `i = 1`
- The function assumes $n \geq 1$, as the pseudo-code does: for $n = 0$ the element `a[0]` does not exist

The C++ implementation (1/3)

```cpp
#include <iostream>

using namespace std;

// Function to find the maximum value in an array
int ArrayMaxValue(int n, int *a)
{
    int max = a[0]; // Initialize with the first element

    for (int i = 1; i < n; i++)
    {
        if (a[i] > max)
        {
            max = a[i]; // Update the maximum
        }
    }

    return max;
}
```

The C++ implementation (2/3)

```
21 int main()
22 {
23     int n = 7; // Size of the array
24
25     int* a = new int[n]; // Memory allocation of an
       array of size n
26
27     int values[] = {9, 12, 3, 16, 19, 11, 12};
28     for (int i = 0; i < n; i++)
29     {
30         a[i] = values[i];
31     }
32
33     int max = ArrayMaxValue(n, a);
34     cout << "Maximum value: " << max << endl;
35
36     delete[] a; // Free allocated memory
37
38     return 0;
39 }
```

The C++ implementation (3/3)

Output:

```
Maximum value: 19
```

- Now we follow the execution step by step, on the instance of the program and on a second instance

Example: step-by-step execution (1/2)

In each table, the column i is the index of the C++ loop and the column $i+1$ is the corresponding index of the pseudo-code. The first row is the initialization `max = a[0]`.

Instance 1: $a = [9, 12, 3, 16, 19, 11, 12]$, $n = 7$

i (C++)	$i+1$ (pseudo-code)	`a[i]`	`a[i] > max?`	`max`
–	–	–	–	9
1	2	12	Yes	12
2	3	3	No	12
3	4	16	Yes	16
4	5	19	Yes	19
5	6	11	No	19
6	7	12	No	19

Return: `max` $= 19$

Example: step-by-step execution (2/2)

Instance 2: $a = [3, 7, 2, 9, 1]$, $n = 5$

i (C++)	$i + 1$ (pseudo-code)	`a[i]`	`a[i] > max?`	`max`
–	–	–	–	3
1	2	7	Yes	7
2	3	2	No	7
3	4	9	Yes	9
4	5	1	No	9

Return: `max` $= 9$

Summary

- The algorithm `FindMaximum` uses a function, a loop, a conditional statement and an array
- Pseudo-code: $max := a[1]$ and the loop from $i := 2$ to n
- C++: `max = a[0]` and the loop from `i = 1` while `i < n`; the entry $a[i + 1]$ of the pseudo-code is `a[i]`
- Both assume $n \geq 1$; on the two instances the maximum is 19 and 9

Bitwise operators

Introduction: bitwise operators

- The integer numbers handled by a program are stored in binary, as described in the chapter «Representation of numbers»
- C++ provides operators that act directly on these binary digits
- We see the six bitwise operators, one by one, and then some practical applications

Bitwise operators

- **Bitwise operators** perform operations on the individual bits of integer values
- They are essential for low-level programming, optimization, and working with binary representations
- The bits are numbered from the right, starting from position 0 (the least significant bit)

The bitwise operators of C++

Bitwise operators in `C++`:

- `&` (Bitwise AND) $\rightsquigarrow$ Returns 1 only if both corresponding bits are 1.
- `|` (Bitwise OR) $\rightsquigarrow$ Returns 1 if at least one corresponding bit is 1.
- `^` (Bitwise XOR) $\rightsquigarrow$ Returns 1 if the corresponding bits are different.
- `~` (Bitwise NOT) $\rightsquigarrow$ Flips all bits (0 becomes 1, 1 becomes 0).
- `<<` (Left shift) $\rightsquigarrow$ Shifts the bits to the left by a specified number of positions.
- `>>` (Right shift) $\rightsquigarrow$ Shifts the bits to the right by a specified number of positions.

Bitwise AND (&)

- The bitwise AND operator compares each bit of two numbers and returns 1 only if both bits are 1
- Truth table for AND:

Bit A	Bit B	A & B
0	0	0
0	1	0
1	0	0
1	1	1

Example: bitwise AND

```cpp
int a = 12;  // Binary: 1100
int b = 10;  // Binary: 1010
int result = a & b;  // Binary: 1000 (decimal: 8)

cout << "a & b = " << result << endl;  // Output: 8
```

Bit-by-bit operation:

					Decimal
a	1	1	0	0	12
b	1	0	1	0	10
a & b	1	0	0	0	8

Bitwise OR (|)

- The bitwise OR operator compares each bit of two numbers and returns 1 if at least one bit is 1
- Truth table for OR:

Bit A	Bit B	A | B
0	0	0
0	1	1
1	0	1
1	1	1

Example: bitwise OR

```cpp
int a = 12;  // Binary: 1100
int b = 10;  // Binary: 1010
int result = a | b;  // Binary: 1110 (decimal: 14)

cout << "a | b = " << result << endl;  // Output: 14
```

Bit-by-bit operation:

					Decimal
	a	1	1	0	0	12
	b	1	0	1	0	10
a	b	1	1	1	0	14

Bitwise XOR (^)

- The bitwise XOR (exclusive OR) operator compares each bit of two numbers and returns 1 if the bits are different
- Truth table for XOR:

Bit A	Bit B	A ^ B
0	0	0
0	1	1
1	0	1
1	1	0

Example: bitwise XOR

```cpp
int a = 12;  // Binary: 1100
int b = 10;  // Binary: 1010
int result = a ^ b;  // Binary: 0110 (decimal: 6)

cout << "a ^ b = " << result << endl;  // Output: 6
```

Bit-by-bit operation:

					Decimal
a	1	1	0	0	12
b	1	0	1	0	10
a ^ b	0	1	1	0	6

Bitwise NOT (~)

- The bitwise NOT operator flips all bits of a number (0 becomes 1, 1 becomes 0)
- This is a unary operator (it operates on a single operand)
- The result depends on the number of bits used to represent the integer
- In the example we use the data type `unsigned char`, which stores the natural numbers from 0 to 255 with 8 bits
- The conversion `(int)` displays the result as a number instead of a character

Example: bitwise NOT

```
1 unsigned char a = 12;  // Binary (8 bits): 00001100
2 unsigned char result = ~a;  // Binary: 11110011 (
      decimal: 243)
3
4 cout << "~a = " << (int)result << endl;  // Output: 243
```

Bit-by-bit operation (using the 8-bit representation):

									Decimal
a	0	0	0	0	1	1	0	0	12
~a	1	1	1	1	0	0	1	1	243

Indeed, $243 = 255 - 12$: flipping all the 8 bits of a number x gives $255 - x$.

Left shift (<<)

- The left shift operator shifts all bits to the left by a specified number of positions
- Zeros are added on the right side
- Shifting left by k positions is equivalent to multiplying by 2^k (as long as the result can be represented by the data type)

Example: left shift

```cpp
int a = 5;  // Binary: 0101
int result = a << 2;  // Shift left by 2 positions:
    10100 (decimal: 20)

cout << "a << 2 = " << result << endl;  // Output: 20
// Equivalent to: 5 * 2^2 = 5 * 4 = 20
```

Bit-by-bit operation:

							Decimal	Equivalent
a	0	0	0	1	0	1	5	5
a << 1	0	0	1	0	1	0	10	5×2^1
a << 2	0	1	0	1	0	0	20	5×2^2
a << 3	1	0	1	0	0	0	40	5×2^3

Right shift (>>)

- The right shift operator shifts all bits to the right by a specified number of positions; the bits that exit on the right are lost
- For non-negative integers, zeros are added on the left side
- For non-negative integers, shifting right by k positions is equivalent to the integer division by 2^k, that is, to $\lfloor x/2^k \rfloor$

Example: right shift (1/2)

```
int a = 20;  // Binary: 10100
int result = a >> 2;  // Shift right by 2 positions:
    00101 (decimal: 5)

cout << "a >> 2 = " << result << endl;  // Output: 5
// Equivalent to: 20 / 2^2 = 20 / 4 = 5
```

Example: right shift (2/2)

Bit-by-bit operation:

							Decimal	Equivalent
a	0	1	0	1	0	0	20	20
a >> 1	0	0	1	0	1	0	10	$20/2^1$
a >> 2	0	0	0	1	0	1	5	$20/2^2$
a >> 3	0	0	0	0	1	0	2	$\lfloor 20/2^3 \rfloor$

In the last row $20/2^3 = 2.5$, and the right shift gives the integer part 2: the bit 1 in position 0 of a >> 2 is lost.

Practical applications

- The bitwise operators give short and fast solutions to common tasks
- In the following examples, a **mask** is a number whose binary representation has the bit 1 only in the positions we want to act on
- The mask `1 << k` has the bit 1 only in position k

Example: checking if a number is even or odd

```
1 int n = 15;
2
3 if (n & 1)
4     cout << n << " is odd" << endl;
5 else
6     cout << n << " is even" << endl;
7
8 // Explanation: n & 1 checks the least significant bit
9 // If it is 1, the number is odd; if it is 0, the
      number is even
```

Output:

```
15 is odd
```

Example: swapping two numbers without a temporary variable (1/2)

```
1 int a = 5, b = 9;
2
3 cout << "Before swap: a = " << a << ", b = " << b <<
      endl;
4
5 a = a ^ b;  // a now holds a XOR b
6 b = a ^ b;  // b = (a XOR b) XOR b = a
7 a = a ^ b;  // a = (a XOR b) XOR a = b
8
9 cout << "After swap: a = " << a << ", b = " << b <<
      endl;
```

Output:

```
Before swap: a = 5, b = 9
After swap: a = 9, b = 5
```

Example: swapping two numbers without a temporary variable (2/2)

The swap works because
`x ^ x` is 0 and `x ^ 0` is `x` for every integer `x`,
and the XOR is commutative and associative.

Example: checking if a specific bit is set

```cpp
int n = 12;  // Binary: 1100
int position = 2;  // Check the bit at position 2 (from right, starting at 0)

if (n & (1 << position))
    cout << "Bit " << position << " is set (1)" << endl;
else
    cout << "Bit " << position << " is not set (0)" << endl;

// Explanation: (1 << position) creates a mask with only bit at position set
// n & mask checks if that bit is 1 in n
```

Output:

```
Bit 2 is set (1)
```

Example: setting a specific bit to 1

```
int n = 12;  // Binary: 1100
int position = 1;  // Set the bit at position 1

n = n | (1 << position);  // Set bit at position 1 to 1

cout << "Result: " << n << endl;  // Output: 14 (Binary
    : 1110)
```

Example: clearing a specific bit to 0

```
1 int n = 14;  // Binary: 1110
2 int position = 1;  // Clear the bit at position 1
3
4 n = n & ~(1 << position);  // Set bit at position 1 to
    0
5
6 cout << "Result: " << n << endl;  // Output: 12 (Binary
    : 1100)
```

The mask `~(1 << position)` has the bit 0 in the given position and the bit 1 in all the other positions.

Example: toggling a specific bit

```
int n = 12;  // Binary: 1100
int position = 0;  // Toggle the bit at position 0

n = n ^ (1 << position);  // Toggle bit at position 0

cout << "Result: " << n << endl;  // Output: 13 (Binary
    : 1101)
```

Summary (1/2)

The six bitwise operators:

- `&` (AND), `|` (OR), `^` (XOR) compare the corresponding bits of two numbers
- `~` (NOT) flips all the bits; the result depends on the number of bits of the data type

Shifts:

- `x << k` multiplies x by 2^k (if the result can be represented)
- For $x \geq 0$, `x >> k` computes $\lfloor x/2^k \rfloor$: the bits that exit on the right are lost

Summary (2/2)

Practical applications:

- `n & 1` tells whether `n` is odd
- Three XOR operations swap two numbers without a temporary variable
- With the mask `1 << k`:
  - `n & mask` checks the bit in position k
  - `n | mask` sets it to 1
  - `n & ~mask` clears it to 0
  - `n ^ mask` toggles it

Key points of the chapter

Introduction: key points of the chapter

- We collect the key points of the chapter
- They are the C++ tools that we use to implement the algorithms of the course

Summary (1/2)

- A `C++` program is compiled (for example with `g++`) and its execution starts from the `main` function; every instruction ends with ";"
- Every variable is declared with its data type before use; the division between two `int` values is the integer division
- Conditional statements (`if`, `else`, `?:`) choose which instructions are executed; loops (`while`, `for`) repeat them

Summary (2/2)

- The arrays of `C++` are indexed from 0 to $n-1$, while the arrays of the pseudo-code are indexed from 1 to n; a dynamic-size array is allocated with `new` and freed with `delete[]`
- A function receives its parameters by value (a copy) or by reference (`&`); an array is passed through the pointer to its first element, together with its size
- The bitwise operators act on the binary representation of the integer numbers: `x << k` multiplies x by 2^k and, for $x \geq 0$, `x >> k` computes $\lfloor x/2^k \rfloor$